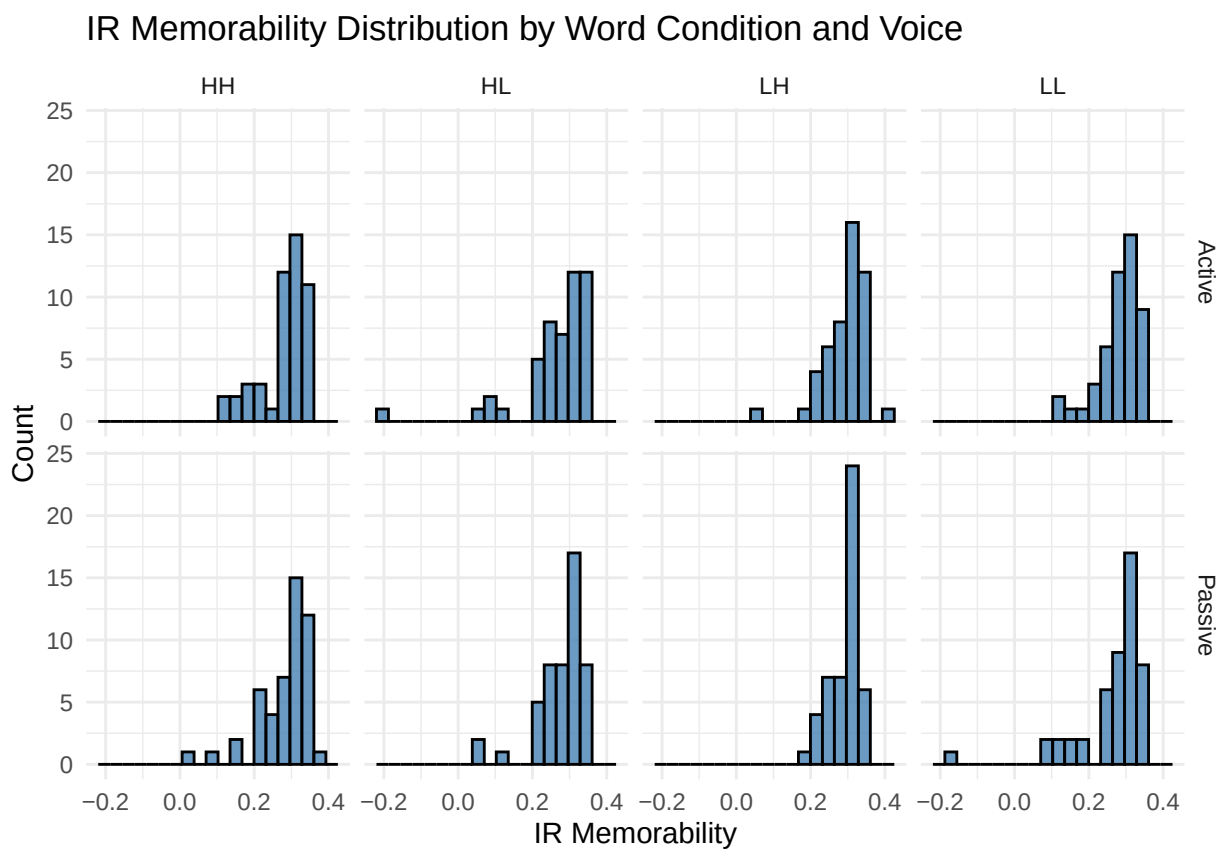
Sentence Memorability Graphs

```r
raw_dataset <- read_csv(
  "sentence_memorability_data.csv",
  na = c("", "NA", "N/A"),
  show_col_types = FALSE
)

aggregated_matrix <- raw_dataset %>%
  filter(isTarget == TRUE) %>%
  arrange(participant_ID, Stimulus, Timestamp) %>%
  mutate(
    # mapping conditions
    Word_Condition = case_when(
  str_detect(Stimulus, "^HH_")  ~ "HH",
  str_detect(Stimulus, "^HVL_") ~ "HL",
  str_detect(Stimulus, "^LVH_") ~ "LH",
  str_detect(Stimulus, "^LVL_") ~ "LL"
),
    Word_Condition = factor(Word_Condition, levels = c("HH","HL","LH","LL")),

    Sentence_Voice = case_when(
      str_detect(Stimulus, "_A$") ~ "Active",
      str_detect(Stimulus, "_P$") ~ "Passive",
      TRUE ~ NA_character_
    ),
    Sentence_Voice = factor(Sentence_Voice, levels = c("Active", "Passive")),

    Accuracy.IR = suppressWarnings(as.numeric(Accuracy.IR)),
    Accuracy.WR = suppressWarnings(as.numeric(Accuracy.WR)),

    # store previous IR result
    prev_IR = lag(Accuracy.IR)
  ) %>%
  group_by(Stimulus, Word_Condition, Sentence_Voice) %>%
  summarise(

    # IR components
    Total_Repeats = sum(isRepeat == TRUE, na.rm = TRUE),
    Total_Denom = sum(is.na(isRepeat) & Event == "Sentence shown"),
    Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy.IR == 1, na.rm = TRUE),
    False_Alarms = sum(Event == "IR pressed" & is.na(isRepeat), na.rm = TRUE),

    # IR metrics
    Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
    False_Alarm_Rate = if_else(Total_Denom > 0, False_Alarms / Total_Denom, 0),
    IR_Memorability = Hit_Rate - False_Alarm_Rate,
```

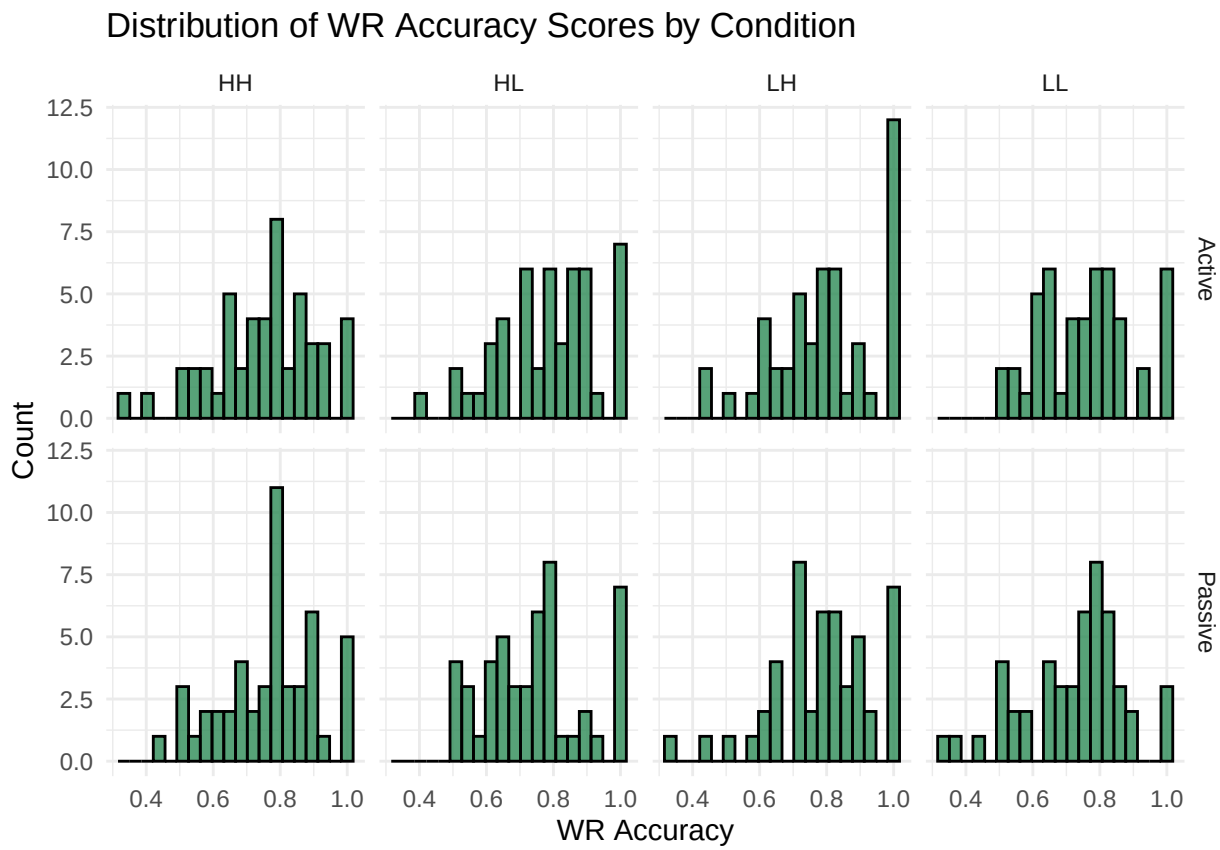
```r
    # WR components (after correct IR)
    WR_correct = sum(Event == "WR pressed" & Accuracy.WR == 1 & prev_IR == 1, na.rm = TRUE),
    WR_total   = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),

    # WR metric
    WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),

    .groups = "drop"
  ) %>%
  mutate(WR_Accuracy = replace_na(WR_Accuracy, 0))
```

```r
# Graph 1: Histogram of IR Memorability Scores
ggplot(aggregated_matrix, aes(x = IR_Memorability)) +
  geom_histogram(bins = 20, fill = "steelblue", color = "black", alpha = 0.8) +
  facet_grid(Sentence_Voice ~ Word_Condition) +
  labs(
    title = "IR Memorability Distribution by Word Condition and Voice",
    x = "IR Memorability",
    y = "Count"
  ) +
  theme_minimal()
```

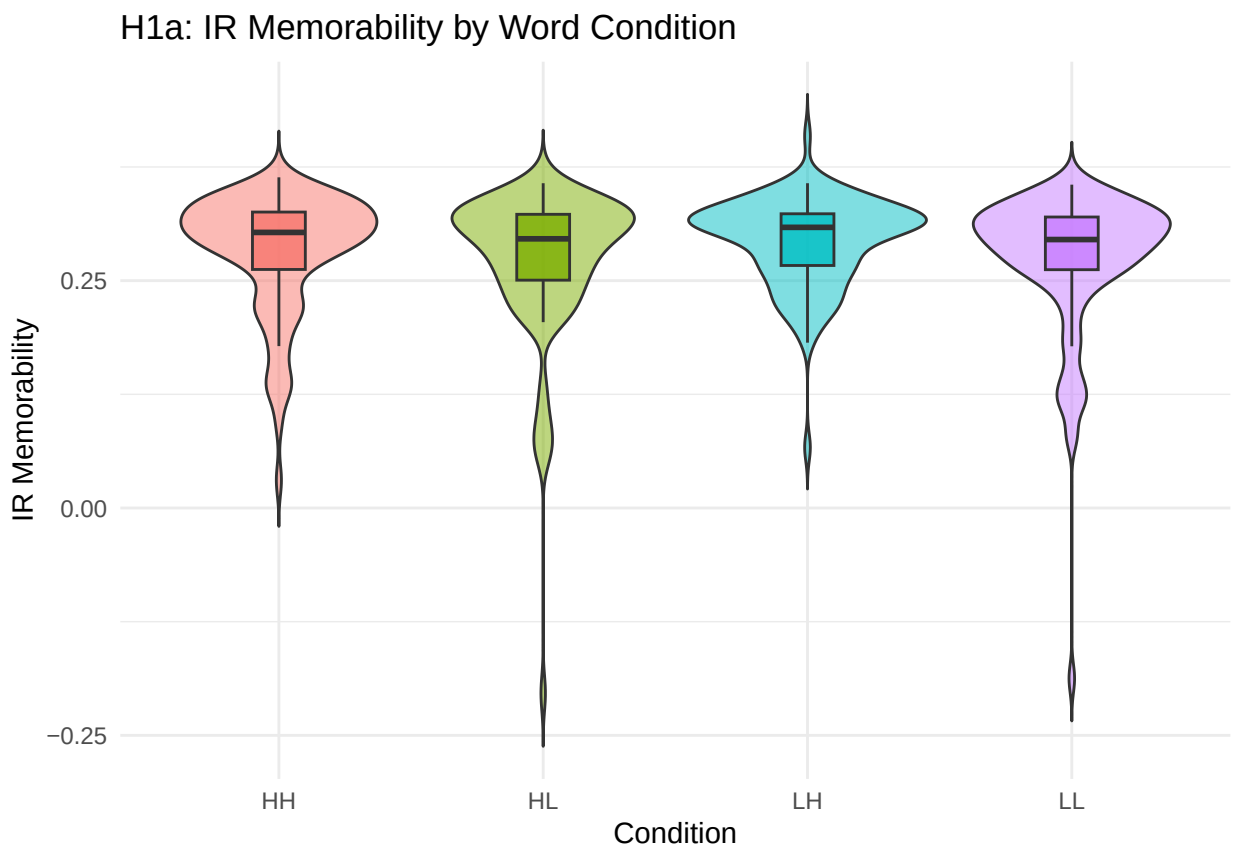



```r
# Graph 2: Histogram of WR Accuracy Scores
ggplot(aggregated_matrix, aes(x = WR_Accuracy)) +
  geom_histogram(bins = 20, fill = "seagreen", color = "black", alpha = 0.8) +
  facet_grid(Sentence_Voice ~ Word_Condition) +
```

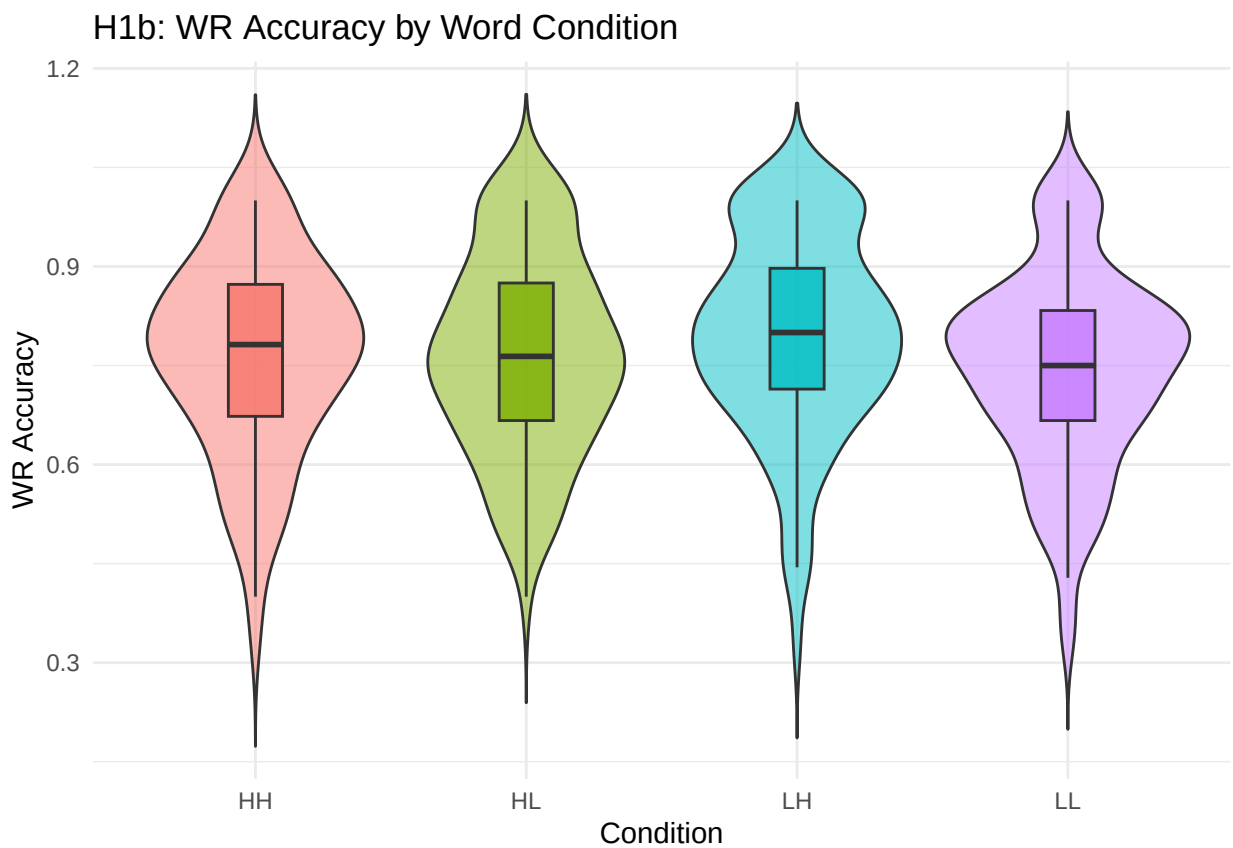
```r
  labs(
    title = "Distribution of WR Accuracy Scores by Condition",
    x = "WR Accuracy",
    y = "Count"
  ) +
  theme_minimal()
```



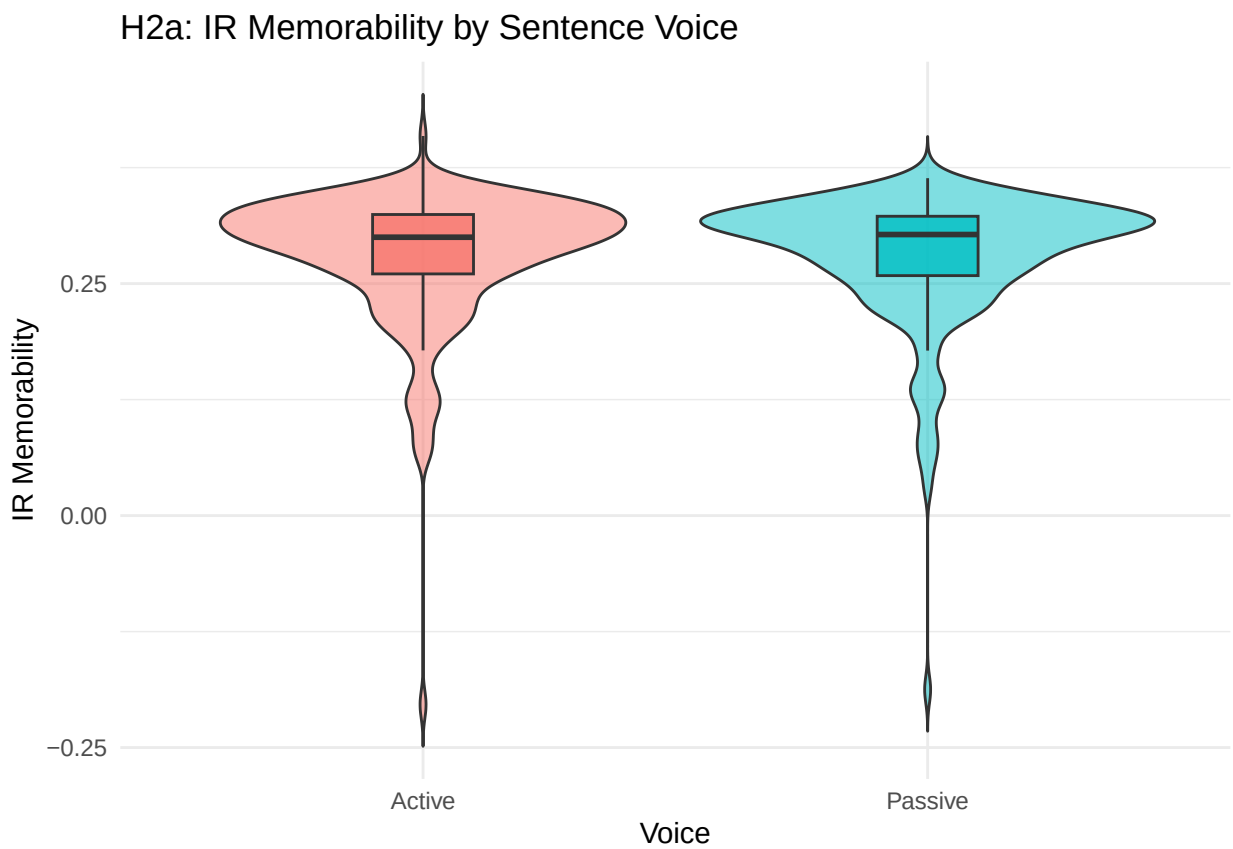

```r
# Graph 3 (H1a): Violin & Boxplot for IR by Word Condition
ggplot(aggregated_matrix, aes(x = Word_Condition, y = IR_Memorability, fill = Word_Condition)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  labs(title = "H1a: IR Memorability by Word Condition", x = "Condition", y = "IR Memorability") +
  theme_minimal() + theme(legend.position = "none")
```



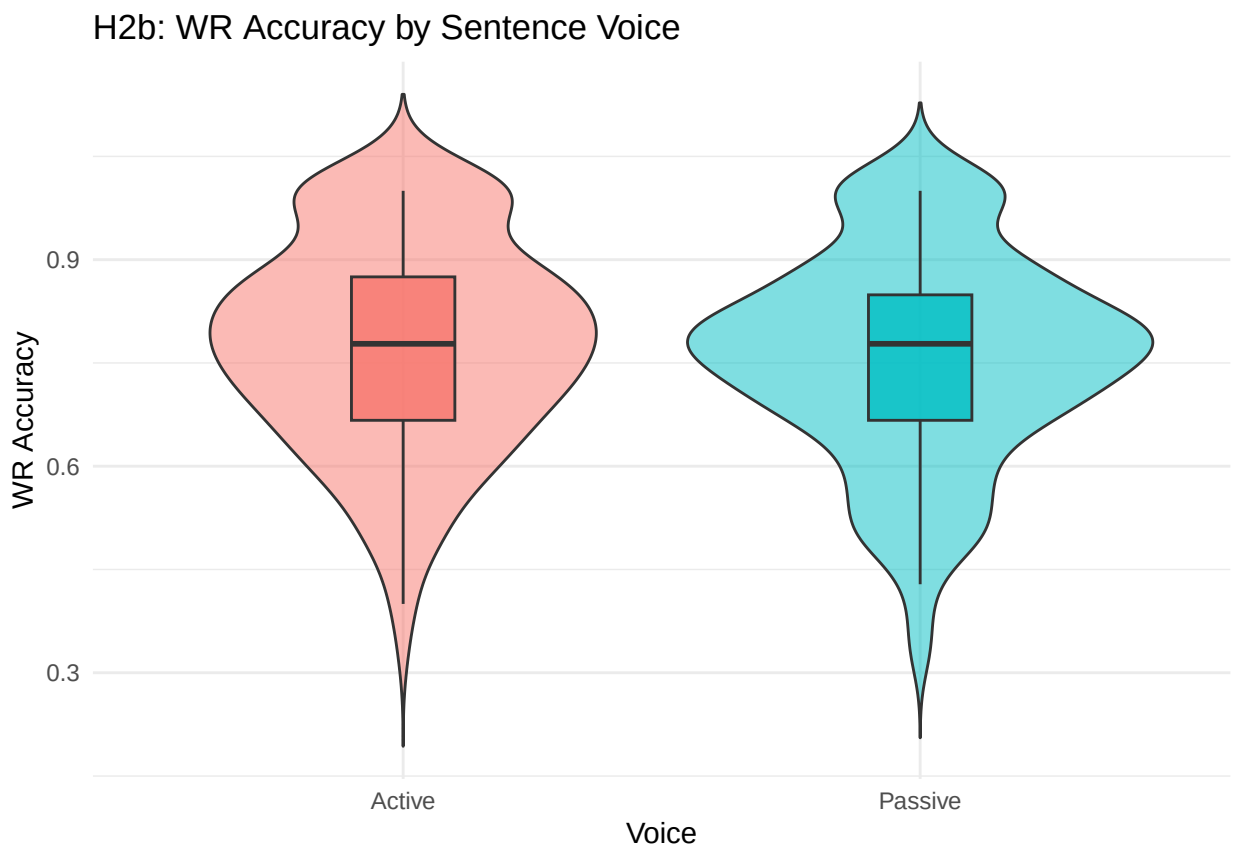

```r
# Graph 4 (H1b): Violin & Boxplot for WR by Word Condition
ggplot(aggregated_matrix, aes(x = Word_Condition, y = WR_Accuracy, fill = Word_Condition)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  labs(title = "H1b: WR Accuracy by Word Condition", x = "Condition", y = "WR Accuracy") +
  theme_minimal() + theme(legend.position = "none")
```



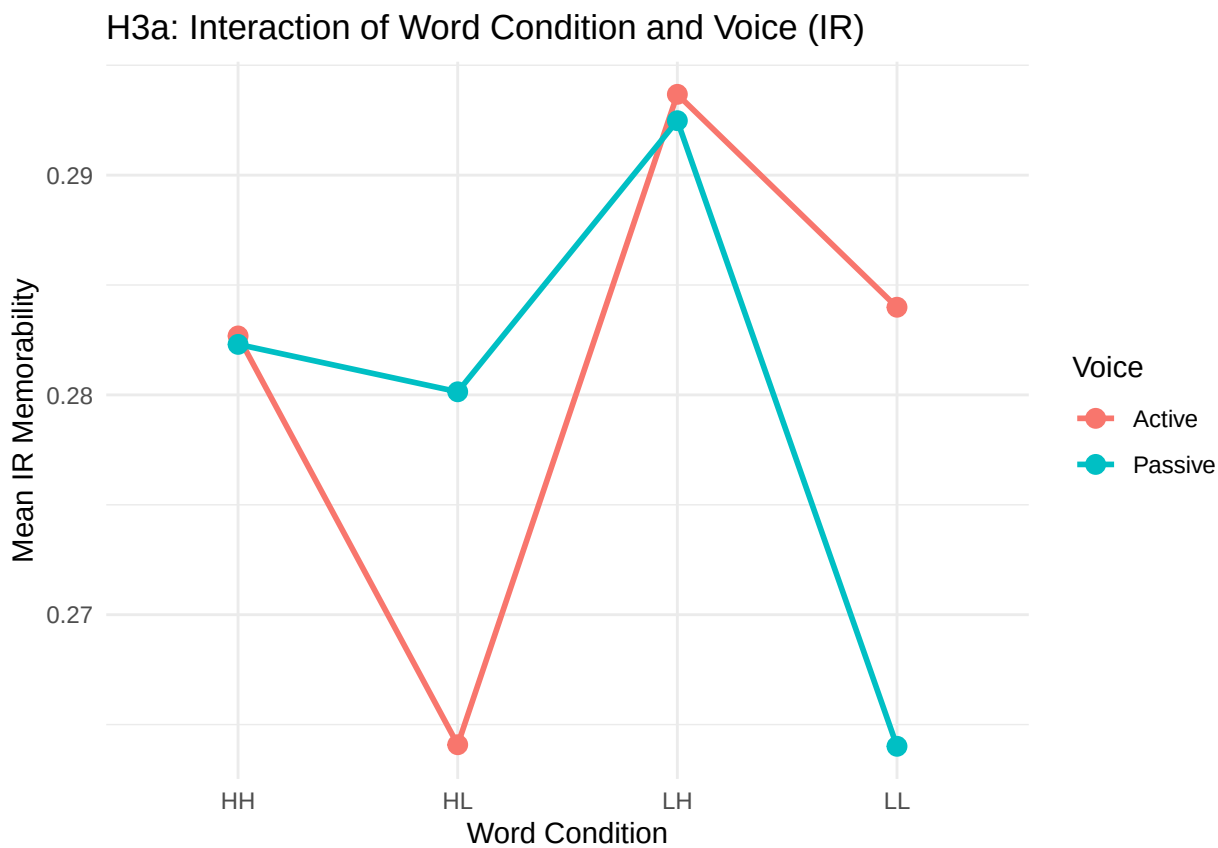

```
# Graph 5 (H2a): Violin & Boxplot for IR by Voice
ggplot(aggregated_matrix %>% drop_na(Sentence_Voice), aes(x = Sentence_Voice, y = IR_Memorability, fill
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  labs(title = "H2a: IR Memorability by Sentence Voice", x = "Voice", y = "IR Memorability") +
  theme_minimal() + theme(legend.position = "none")
```



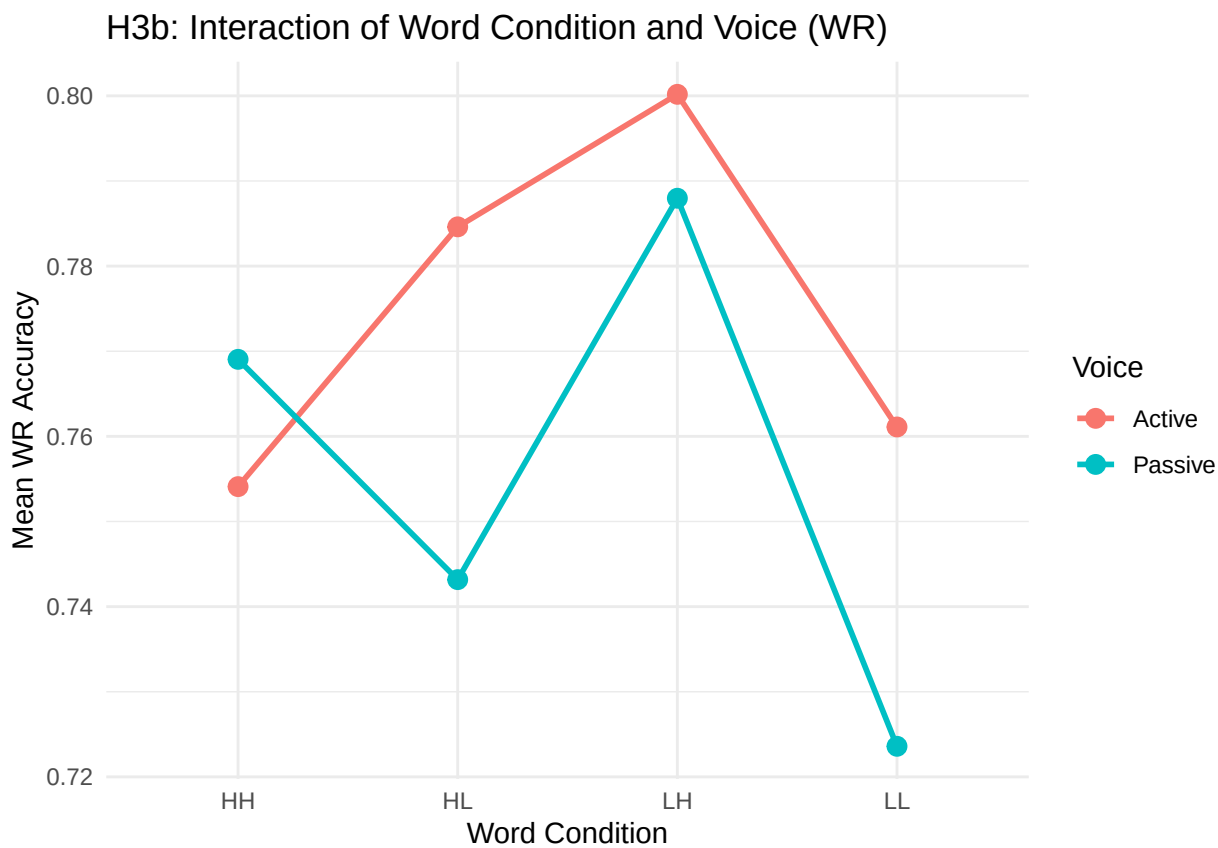

```
# Graph 6 (H2b): Violin & Boxplot for WR by Voice
ggplot(aggregated_matrix %>% drop_na(Sentence_Voice), aes(x = Sentence_Voice, y = WR_Accuracy, fill = S
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  labs(title = "H2b: WR Accuracy by Sentence Voice", x = "Voice", y = "WR Accuracy") +
  theme_minimal() + theme(legend.position = "none")
```




```
interaction_data <- aggregated_matrix %>%
  drop_na(Word_Condition, Sentence_Voice) %>%
  group_by(Word_Condition, Sentence_Voice) %>%
  summarise(
    Mean_IR = mean(IR_Memorability, na.rm = TRUE),
    Mean_WR = mean(WR_Accuracy, na.rm = TRUE),
    .groups = 'drop'
  )

# Graph 7 (H3a): Interaction Plot for IR
ggplot(interaction_data, aes(x = Word_Condition, y = Mean_IR, color = Sentence_Voice, group = Sentence_
  geom_line(linewidth = 1) + geom_point(size = 3) +
  labs(title = "H3a: Interaction of Word Condition and Voice (IR)", x = "Word Condition", y = "Mean IR
  theme_minimal()
```

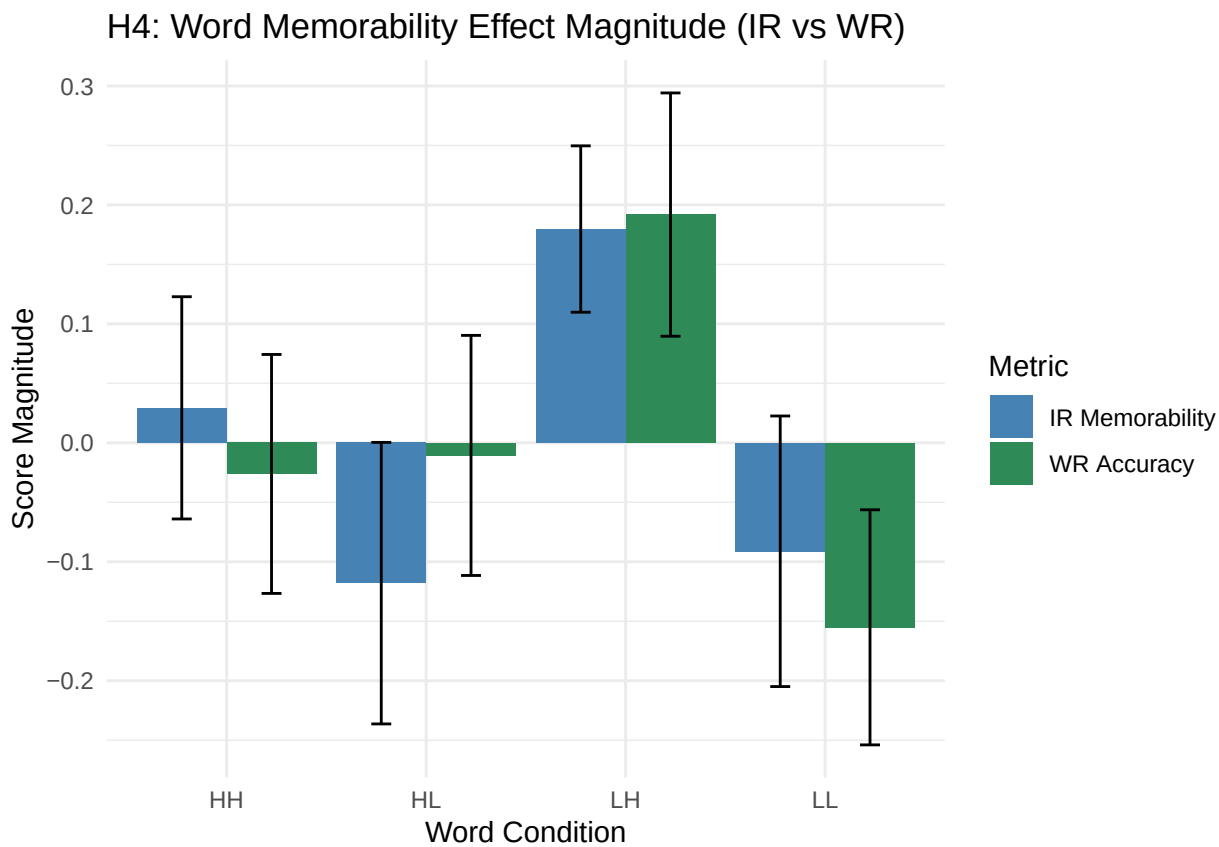



```r
# Graph 8 (H3b): Interaction Plot for WR
ggplot(interaction_data, aes(x = Word_Condition, y = Mean_WR, color = Sentence_Voice, group = Sentence_V
  geom_line(linewidth = 1) + geom_point(size = 3) +
  labs(title = "H3b: Interaction of Word Condition and Voice (WR)", x = "Word Condition", y = "Mean WR A
  theme_minimal()
```

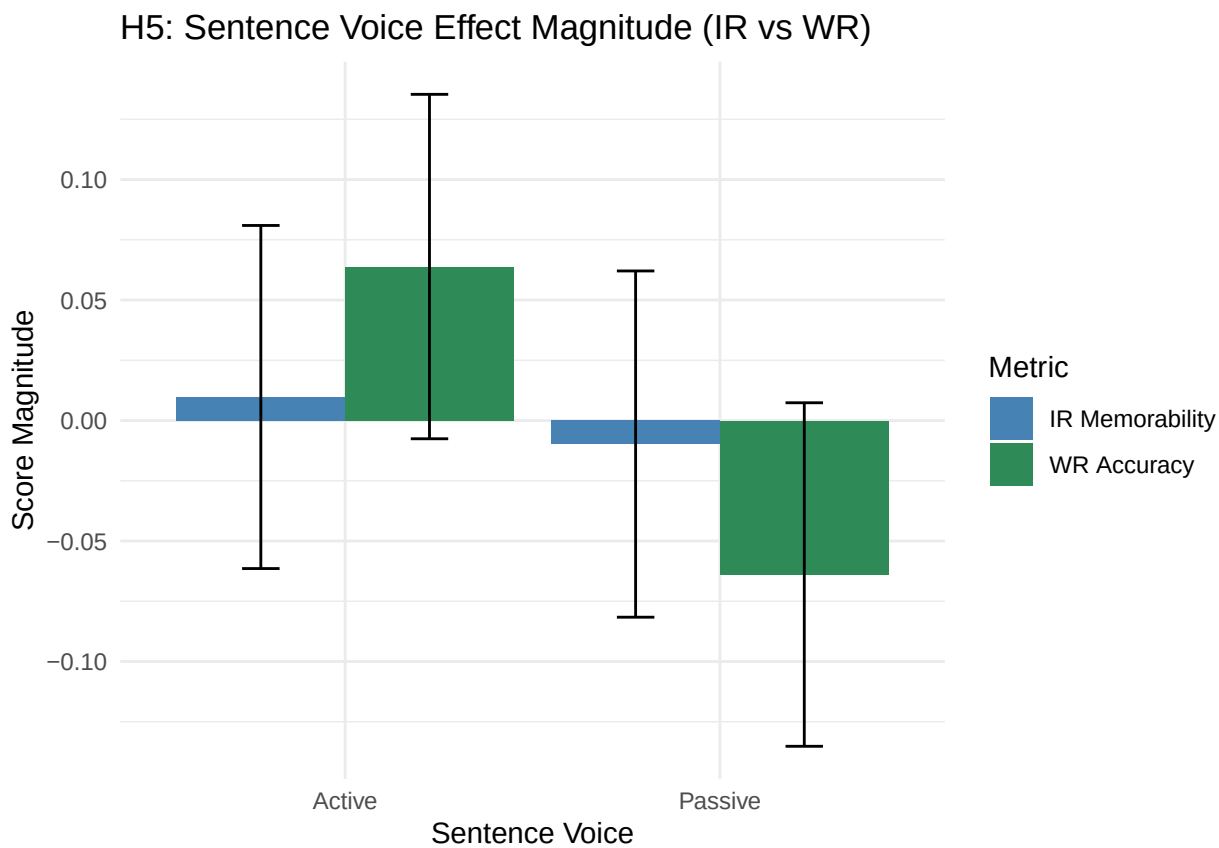



```r
aggregated_scaled <- aggregated_matrix %>%
  mutate(
    IR_scaled = as.numeric(scale(IR_Memorability)),
    WR_scaled = as.numeric(scale(WR_Accuracy))
  )

# Calculate average scaled IR and WR scores and their standard errors for each word condition
effect_data <- aggregated_scaled %>%
  drop_na(Word_Condition, Sentence_Voice) %>%
  group_by(Word_Condition) %>%
  summarise(
    IR_mean = mean(IR_scaled, na.rm=TRUE),
    IR_se = sd(IR_scaled, na.rm=TRUE)/sqrt(n()),
    WR_mean = mean(WR_scaled, na.rm=TRUE),
    WR_se = sd(WR_scaled, na.rm=TRUE)/sqrt(n()),
    .groups="drop"
  )

# Graph 9 (H4): Word Memorability effects on IR vs WR
effect_data_long <- effect_data %>%
  pivot_longer(cols = c(IR_mean, WR_mean), names_to = "Metric", values_to = "Mean") %>%
  mutate(SE = ifelse(Metric == "IR_mean", IR_se, WR_se))

ggplot(effect_data_long, aes(x = Word_Condition, y = Mean, fill = Metric)) +
  geom_bar(stat = "identity", position = position_dodge()) +
  geom_errorbar(aes(ymin = Mean - SE, ymax = Mean + SE), width = 0.2, position = position_dodge(0.9)) +
```

```
  labs(title = "H4: Word Memorability Effect Magnitude (IR vs WR)", x = "Word Condition", y = "Score Mag
  scale_fill_manual(values = c("steelblue", "seagreen"), labels = c("IR Memorability", "WR Accuracy")) +
  theme_minimal()
```




```
# Graph 10 (H5): Sentence Voice effects on IR vs WR
voice_effect <- aggregated_scaled %>%
  drop_na(Sentence_Voice) %>%
  group_by(Sentence_Voice) %>%
  summarise(
    IR_mean = mean(IR_scaled, na.rm=TRUE),
    IR_se = sd(IR_scaled, na.rm=TRUE)/sqrt(n()),
    WR_mean = mean(WR_scaled, na.rm=TRUE),
    WR_se = sd(WR_scaled, na.rm=TRUE)/sqrt(n()),
    .groups="drop"
  ) %>%
  pivot_longer(cols = c(IR_mean, WR_mean), names_to = "Metric", values_to = "Mean") %>%
  mutate(SE = ifelse(Metric == "IR_mean", IR_se, WR_se))

ggplot(voice_effect, aes(x = Sentence_Voice, y = Mean, fill = Metric)) +
  geom_bar(stat = "identity", position = position_dodge()) +
  geom_errorbar(aes(ymin = Mean - SE, ymax = Mean + SE), width = 0.2, position = position_dodge(0.9)) +
  labs(title = "H5: Sentence Voice Effect Magnitude (IR vs WR)", x = "Sentence Voice", y = "Score Magni
  scale_fill_manual(values = c("steelblue", "seagreen"), labels = c("IR Memorability", "WR Accuracy")) +
  theme_minimal()
```




```r
# Descriptive stats by Word Condition
desc_word <- aggregated_matrix %>%
  drop_na(Word_Condition) %>%
  group_by(Word_Condition) %>%
  summarise(
    IR_mean = mean(IR_Memorability, na.rm = TRUE),
    IR_sd = sd(IR_Memorability, na.rm = TRUE),
    IR_median = median(IR_Memorability, na.rm = TRUE),

    WR_mean = mean(WR_Accuracy, na.rm = TRUE),
    WR_sd = sd(WR_Accuracy, na.rm = TRUE),
    WR_median = median(WR_Accuracy, na.rm = TRUE),

    n = n(),
    .groups = "drop"
  )

print(desc_word)
```

```
## # A tibble: 4 x 8
##   Word_Condition IR_mean  IR_sd IR_median WR_mean WR_sd WR_median     n
##   <fct>            <dbl>  <dbl>     <dbl>   <dbl> <dbl>     <dbl> <int>
## 1 HH               0.282 0.0651     0.303   0.762 0.148     0.782    98
## 2 HL               0.272 0.0825     0.296   0.764 0.149     0.764    98
## 3 LH               0.293 0.0488     0.309   0.794 0.151     0.8      98
## 4 LL               0.274 0.0793     0.295   0.742 0.146     0.75     98
```

```r
# Descriptive stats by Sentence Voice
desc_voice <- aggregated_matrix %>%
  drop_na(Sentence_Voice) %>%
  group_by(Sentence_Voice) %>%
  summarise(
    IR_mean = mean(IR_Memorability, na.rm = TRUE),
    IR_sd = sd(IR_Memorability, na.rm = TRUE),
    IR_median = median(IR_Memorability, na.rm = TRUE),

    WR_mean = mean(WR_Accuracy, na.rm = TRUE),
    WR_sd = sd(WR_Accuracy, na.rm = TRUE),
    WR_median = median(WR_Accuracy, na.rm = TRUE),

    n = n(),
    .groups = "drop"
  )

print(desc_voice)
```

```
## # A tibble: 2 x 8
##   Sentence_Voice IR_mean  IR_sd IR_median WR_mean WR_sd WR_median     n
##   <fct>            <dbl>  <dbl>     <dbl>   <dbl> <dbl>     <dbl> <int>
## 1 Active           0.281 0.0702     0.3     0.775 0.149     0.778   196
## 2 Passive          0.280 0.0708     0.303   0.756 0.149     0.778   196
```

```r
kruskal_ir_word <- kruskal.test(IR_Memorability ~ Word_Condition, data = aggregated_matrix)
print(kruskal_ir_word)
```

```
##
##  Kruskal-Wallis rank sum test
##
## data:  IR_Memorability by Word_Condition
## Kruskal-Wallis chi-squared = 2.623, df = 3, p-value = 0.4535
```

```r
kruskal_wr_word <- kruskal.test(WR_Accuracy ~ Word_Condition, data = aggregated_matrix)
print(kruskal_wr_word)
```

```
##
##  Kruskal-Wallis rank sum test
##
## data:  WR_Accuracy by Word_Condition
## Kruskal-Wallis chi-squared = 6.194, df = 3, p-value = 0.1025
```

```r
mw_ir_voice <- wilcox.test(
  IR_Memorability ~ Sentence_Voice,
  data = aggregated_matrix,
  exact = FALSE
)

print(mw_ir_voice)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  IR_Memorability by Sentence_Voice
## W = 19345, p-value = 0.9031
```

```
## alternative hypothesis: true location shift is not equal to 0
```

```r
mw_wr_voice <- wilcox.test(
  WR_Accuracy ~ Sentence_Voice,
  data = aggregated_matrix,
  exact = FALSE
)

print(mw_wr_voice)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  WR_Accuracy by Sentence_Voice
## W = 20505, p-value = 0.2468
## alternative hypothesis: true location shift is not equal to 0
```

```r
# Create categorical correctness variables
fisch_data <- aggregated_matrix %>%
  mutate(
    IR_Correct = ifelse(IR_Memorability > 0, "Correct", "Incorrect"),
    WR_Correct = ifelse(WR_Accuracy > 0.5, "Correct", "Incorrect")
  )

ir_word_table <- table(fisch_data$Word_Condition, fisch_data$IR_Correct)
chisq_ir_word <- fisher.test(ir_word_table)
print(chisq_ir_word)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  ir_word_table
## p-value = 1
## alternative hypothesis: two.sided
```

```r
wr_word_table <- table(fisch_data$Word_Condition, fisch_data$WR_Correct)
chisq_wr_word <- fisher.test(wr_word_table)
print(chisq_wr_word)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  wr_word_table
## p-value = 0.919
## alternative hypothesis: two.sided
```

```r
ir_voice_table <- table(fisch_data$Sentence_Voice, fisch_data$IR_Correct)
chisq_ir_voice <- fisher.test(ir_voice_table)
print(chisq_ir_voice)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  ir_voice_table
## p-value = 1
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
```

```
##   0.01268257 78.84838263
## sample estimates:
## odds ratio
##          1
```

```r
wr_voice_table <- table(fisch_data$Sentence_Voice, fisch_data$WR_Correct)
chisq_wr_voice <- fisher.test(wr_voice_table)
print(chisq_wr_voice)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  wr_voice_table
## p-value = 0.3423
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##  0.6831186 3.6348693
## sample estimates:
## odds ratio
##   1.548831
```

```r
# Create categorical correctness variables
chi_data <- aggregated_matrix %>%
  mutate(
    IR_Correct = ifelse(IR_Memorability > 0, "Correct", "Incorrect"),
    WR_Correct = ifelse(WR_Accuracy > 0.5, "Correct", "Incorrect")
  )

# Perform Chi-square tests to observe whether correctness (IR and WR) is associated with word condition
ir_word_table <- table(chi_data$Word_Condition, chi_data$IR_Correct)
chisq_ir_word <- chisq.test(ir_word_table)
```

```
## Warning in chisq.test(ir_word_table): Chi-squared approximation may be
## incorrect
```

```r
print(chisq_ir_word)
```

```
##
##  Pearson's Chi-squared test
##
## data:  ir_word_table
## X-squared = 2.0103, df = 3, p-value = 0.5703
```

```r
wr_word_table <- table(chi_data$Word_Condition, chi_data$WR_Correct)
chisq_wr_word <- chisq.test(wr_word_table)
print(chisq_wr_word)
```

```
##
##  Pearson's Chi-squared test
##
## data:  wr_word_table
## X-squared = 0.72192, df = 3, p-value = 0.868
```

```r
ir_voice_table <- table(chi_data$Sentence_Voice, chi_data$IR_Correct)
chisq_ir_voice <- chisq.test(ir_voice_table)
```

```
## Warning in chisq.test(ir_voice_table): Chi-squared approximation may be
## incorrect
```

```r
print(chisq_ir_voice)
```

```
##
##  Pearson's Chi-squared test
##
## data:  ir_voice_table
## X-squared = 0, df = 1, p-value = 1
```

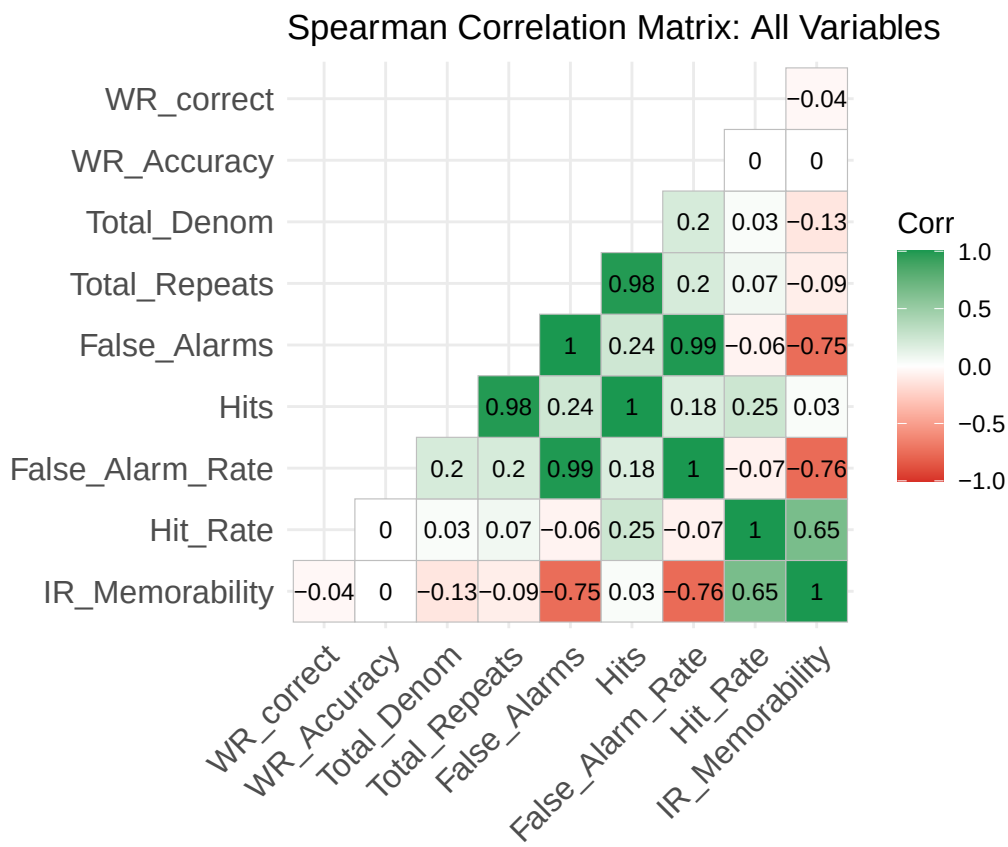
```r
wr_voice_table <- table(chi_data$Sentence_Voice, chi_data$WR_Correct)
chisq_wr_voice <- chisq.test(wr_voice_table)
print(chisq_wr_voice)
```

```
##
##  Pearson's Chi-squared test with Yates' continuity correction
##
## data:  wr_voice_table
## X-squared = 0.90239, df = 1, p-value = 0.3421
```

```r
library(tidyverse)
library(ggcorrplot)

library(dplyr)
library(ggcorrplot)

#  The complete numeric correlation data
cor_data <- aggregated_matrix %>%
  select(
    IR_Memorability,
    WR_Accuracy,
    Hit_Rate,
    False_Alarm_Rate,
    Hits,
    False_Alarms,
    Total_Repeats,
    Total_Denom,
    WR_correct,
    WR_total
  )

# Variable order
var_order <- c(
  "IR_Memorability",
  "Hit_Rate",
  "False_Alarm_Rate",
  "Hits",
  "False_Alarms",
  "Total_Repeats",
  "Total_Denom",
  "WR_Accuracy",
  "WR_correct",
  "WR_total"
)

# Compute Spearman correlation matrix
cor_matrix <- cor(cor_data, method = "spearman", use = "complete.obs")
```

```r
# Reorder matrix
cor_matrix <- cor_matrix[var_order, var_order]

# Reorder correlation matrix and visualize pairwise relationships between variables using a Spearman co
cor_matrix_plot <- cor_matrix[rev(rownames(cor_matrix)), ]

ggcorrplot(
  cor_matrix_plot,
  method = "square",
  type = "lower",
  lab = TRUE,
  lab_size = 3,
  hc.order = FALSE,
  colors = c("#d73027", "white", "#1a9850"),
  title = "Spearman Correlation Matrix: All Variables",
  ggtheme = theme_minimal()
)
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the ggcorrplot package.
##   Please report the issue at <https://github.com/kassambara/ggcorrplot/issues>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```r
ir_wr_counts <- raw_dataset %>%
  summarise(
    IR_Total = sum(Event == "IR pressed", na.rm = TRUE),
    IR_Correct = sum(Event == "IR pressed" & Accuracy.IR == 1, na.rm = TRUE),
    IR_Incorrect = sum(Event == "IR pressed" & Accuracy.IR == 0, na.rm = TRUE),

    WR_Total = sum(Event == "WR pressed", na.rm = TRUE),
    WR_Correct = sum(Event == "WR pressed" & Accuracy.WR == 1, na.rm = TRUE),
    WR_Incorrect = sum(Event == "WR pressed" & Accuracy.WR == 0, na.rm = TRUE)
  )

print(ir_wr_counts)
```

```
## # A tibble: 1 x 6
##   IR_Total IR_Correct IR_Incorrect WR_Total WR_Correct WR_Incorrect
##      <int>      <int>        <int>    <int>      <int>        <int>
## 1     4508       4317          191     4282       3211         1071
```

Contributions:

Bhavani: Data processing and formulae Nishita: Plotting Graphs and Debugging Tejasvini: Organizing the graphs PLotting Graphs